

AI/ML INTERNSHIP – DAY 2 SUBMISSION

Name: Praneetha AS (aspraneetha29@gmail.com)

Role: AI/ML Intern

Date: 12/05/26

Project Folder: Raritone

1. Objective

The objective of Day 2 was to set up a complete Artificial Intelligence environment and understand the fundamentals of human body tracking and pose detection using computer vision techniques. The task involved installing the required tools, studying key concepts such as human pose landmarks and body measurement basics, and implementing a working pose detection system using MediaPipe.

2. Tasks Overview

The following tasks were completed as part of Day 2:

2.1 Installation

- Python
- TensorFlow
- OpenCV
- MediaPipe

2.2 Learning

- Human pose landmarks
- Body measurement basics

2.3 Practice

- Implemented and executed a MediaPipe pose detection demo

2.4 Deliverable

- Generated screenshot of working real-time pose detection system

3. Environment Setup and Installation

A clean virtual environment was created to avoid dependency conflicts and ensure smooth execution of all required libraries.

```
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\Users\aspra>cd Desktop\Raritone

C:\Users\aspra\Desktop\Raritone>python -m venv mp-final

C:\Users\aspra\Desktop\Raritone>mp-final\Scripts\activate
```

Installation Commands

```
pip install tensorflow
pip install opencv-python
pip install mediapipe
```

```
(mp-final) C:\Users\aspra\Desktop\Raritone>pip install tensorflow
Collecting tensorflow
  Using cached tensorflow-2.21.0-cp311-cp311-win_amd64.whl.metadata (4.5 kB)
Requirement already satisfied: absl-py>=1.0.0 in .\mp-final\Lib\site-packages (from tensorflow) (2.4.0)
Collecting astunparse>=1.6.0 (from tensorflow)
```

```
(mp-final) C:\Users\aspra\Desktop\Raritone>pip install mediapipe==0.10.13 opencv-python
Collecting mediapipe==0.10.13
  Downloading mediapipe-0.10.13-cp311-cp311-win_amd64.whl.metadata (9.9 kB)
Requirement already satisfied: opencv-python in .\mp-final\Lib\site-packages (4.13.0.92)
Requirement already satisfied: absl-py in .\mp-final\Lib\site-packages (from mediapipe==0.10.13) (2.4.0)
Collecting attrs>=19.1.0 (from mediapipe==0.10.13)
  Using cached attrs-26.1.0-py3-none-any.whl.metadata (8.8 kB)
Requirement already satisfied: flatbuffers>=2.0 in .\mp-final\Lib\site-packages (from mediapipe==0.10.13) (25.12.19)
Collecting jax (from mediapipe==0.10.13)
  Using cached jax-0.10.0-py3-none-any.whl.metadata (13 kB)
```

Verification

All libraries were verified successfully by importing them in Python without errors:

```
import tensorflow as tf
import cv2
import mediapipe as mp
```

```
(mp-final) C:\Users\aspra\Desktop\Raritone>python --version
Python 3.11.9

(mp-final) C:\Users\aspra\Desktop\Raritone>python -c "import tensorflow as tf; print(tf.__version__)"
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1778566763.151769 13536 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1778566786.792728 13536 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
2.21.0

(mp-final) C:\Users\aspra\Desktop\Raritone>python -c "import cv2; print(cv2.__version__)"
4.13.0

(mp-final) C:\Users\aspra\Desktop\Raritone>python -c "import mediapipe as mp; print(hasattr(mp, 'solutions'))"
AttributeError: 'MessageFactory' object has no attribute 'GetPrototype'
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1778566799.489852 22404 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1778566801.641040 22404 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable 'TF_ENABLE_ONEDNN_OPTS=0'.
True
```

The successful execution of these commands confirmed that the environment was properly configured.

4. Understanding Human Pose Detection

Overview

Pose detection is a computer vision technique used to detect and track human body positions from images or video. It identifies key body points and forms a skeletal structure, enabling machines to understand posture and movement.

How Pose Detection Works

The process, Captures image/video, converts frames to RGB, detects a human, identifies key body landmarks, connects them into a skeleton, and tracks movement across frames.

5. Human Pose Landmarks

Pose landmarks are key points on the body used for tracking movement.

Examples

- Nose, Shoulders, Elbows, Wrists, Hips, Knees, Ankles

Importance

- Helps in posture analysis
- Enables gesture recognition
- Supports real-time movement tracking

6. Body Measurement Basics

Body measurement using AI estimates body proportions based on detected landmarks.

Key Idea

- Distances between body points are calculated
- Ratios are used to estimate proportions

Applications

- Virtual try-on systems
- Fitness tracking
- Clothing size recommendation

Advantages of Pose Detection

- Real-time processing
- High accuracy in detecting body movements
- No need for external sensors
- Useful across multiple domains such as fitness, healthcare, gaming, and fashion

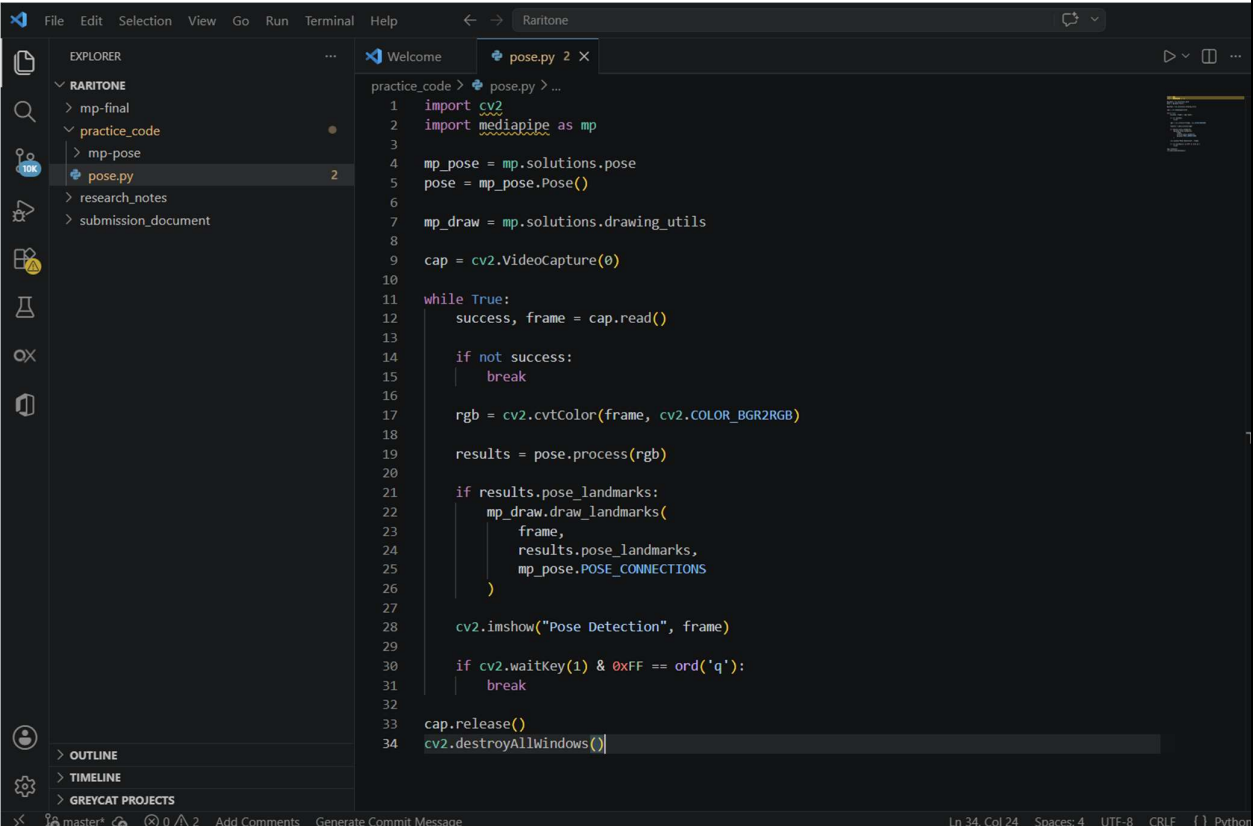
7. Practical Implementation

A real-time pose detection system was implemented using OpenCV and MediaPipe.

Steps Performed

- Accessed webcam using OpenCV
- Captured live video frames
- Converted frames from BGR to RGB
- Processed frames using MediaPipe Pose model
- Detected body landmarks
- Displayed skeletal connections in real time

8. Code Implementation



```
File Edit Selection View Go Run Terminal Help
Raritone
EXPLORER
RARITONE
  mp-final
  practice_code
    mp-pose
    pose.py 2
  research_notes
  submission_document
OUTLINE
TIMELINE
GREYCAT PROJECTS
Welcome
pose.py 2 X
practice_code > pose.py > ...
1 import cv2
2 import mediapipe as mp
3
4 mp_pose = mp.solutions.pose
5 pose = mp_pose.Pose()
6
7 mp_draw = mp.solutions.drawing_utils
8
9 cap = cv2.VideoCapture(0)
10
11 while True:
12     success, frame = cap.read()
13
14     if not success:
15         break
16
17     rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
18
19     results = pose.process(rgb)
20
21     if results.pose_landmarks:
22         mp_draw.draw_landmarks(
23             frame,
24             results.pose_landmarks,
25             mp_pose.POSE_CONNECTIONS
26         )
27
28     cv2.imshow("Pose Detection", frame)
29
30     if cv2.waitKey(1) & 0xFF == ord('q'):
31         break
32
33 cap.release()
34 cv2.destroyAllWindows()
```

9. Output / Deliverable

The pose detection system was successfully executed and produced the following results:

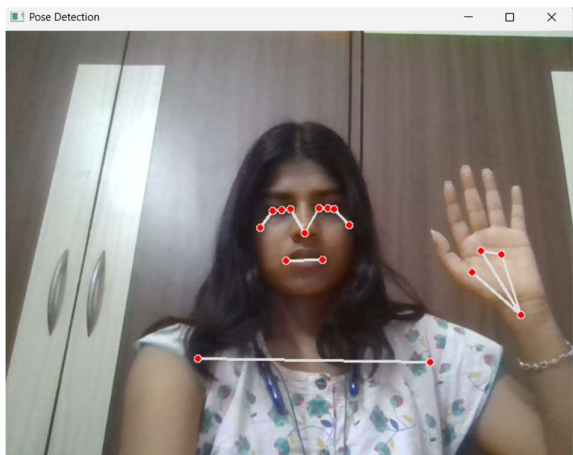
- Real-time webcam video capture
- Detection of human body landmarks
- Display of skeletal structure
- Dynamic tracking of movement

```

PROBLEMS 2 OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell - practice_code + v [] [] ... | [] [] X

PS C:\Users\aspra\Desktop\Raritone> mp-final\Scripts\activate
(mp-final) PS C:\Users\aspra\Desktop\Raritone> cd practice_code
(mp-final) PS C:\Users\aspra\Desktop\Raritone\practice_code> python pose.py
Open file in editor (ctrl + click) te XNNPACK delegate for CPU.
C:\Users\aspra\Desktop\Raritone\mp-final\Lib\site-packages\google\protobuf\symbol_database.py:55: UserWarning: SymbolDatabase.GetPrototype() is deprecated. Please use message_factory.GetMessageClass() instead. SymbolDatabase.GetPrototype() will be removed soon.
  warnings.warn('SymbolDatabase.GetPrototype() is deprecated. Please use
Traceback (most recent call last):
  File "C:\Users\aspra\Desktop\Raritone\practice_code\pose.py", line 19, in <module>
    results = pose.process(rgb)
    ~~~~~
  File "C:\Users\aspra\Desktop\Raritone\mp-final\Lib\site-packages\mediapipe\python\solutions\pose.py", line 185, in process
    results = super().process(input_data={'image': image})
    ~~~~~
  File "C:\Users\aspra\Desktop\Raritone\mp-final\Lib\site-packages\mediapipe\python\solution_base.py", line 340, in process

```



10. Challenges Faced

- Initial installation and dependency conflicts
- MediaPipe compatibility issues
- Incorrect environment usage
- Difficulty in detecting pose due to camera positioning

Solutions

- Created a clean virtual environment
- Installed compatible library versions
- Ensured correct environment activation
- Adjusted camera positioning and lighting

11. Conclusion

This task provided a comprehensive understanding of pose detection and human body tracking using AI and computer vision. By setting up a clean development environment and implementing a real-time pose detection system, both theoretical concepts and practical skills were strengthened.

The use of pose landmarks and body measurement fundamentals demonstrated how machines can interpret human posture and movement effectively. Overall, this task served as a strong foundation for further exploration in AI, computer vision, and real-world applications.